組織追加ダイアログUI設計書

メタデータ

title: 組織追加ダイアログUI設計書
category: UI設計
version: 1.0.0
last_updated: 2026-02-19
author: Development Team
tags: [provider, organization, dialog, create]

関連ドキュメント

- 組織管理メイン画面
- 組織編集ダイアログ
- 組織管理API仕様
- organizationテーブルDDL
- UIポリシー

---

1. 画面レイアウト

1.1. 画面イメージ


組織追加
組織情報
* 組織CD
maiml_lab_001
* 組織表示名
MAIML研究所
* 組織e-mail
sample@sample.co.jp
会社名
会社名を選択してください
* 契約容量
契約容量を入力してください
* 契約ユーザー数
契約ユーザー数を入力してください
* 契約プロジェクト数
契約プロジェクト数を入力してください
* 二段階認証判定
なし
* みなし進捗使用フラグ
あり
* 状態
アクティブ
代表者情報
* 代表システム管理者
サンプル　太郎
* 代表者メールアドレス
sample@sample.co.jp
キャンセル
保存

2. 概要

2.1. 目的

プロバイダーが新しい組織を登録するためのダイアログを定義します。組織CD、組織表示名、組織e-mail、代表システム管理者情報、会社名、二段階認証判定、アクティブ状態を入力して組織マスタに追加します。

2.2. 利用権限

- **Provider（プロバイダー）**: 使用可能

---

3. ダイアログ構成

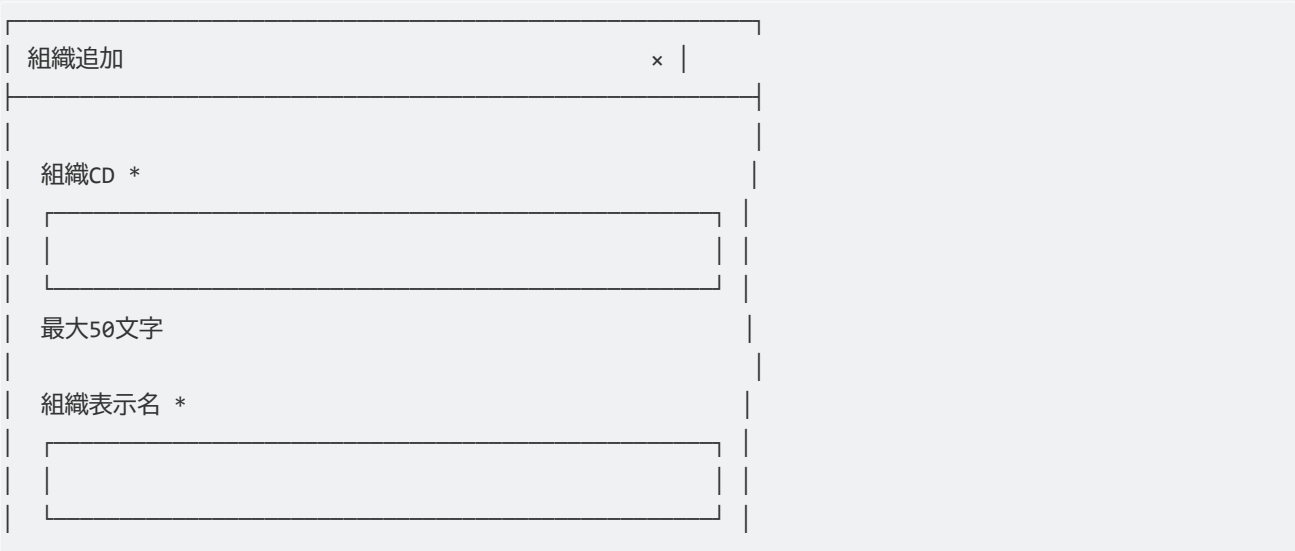
3.1. ダイアログ概要

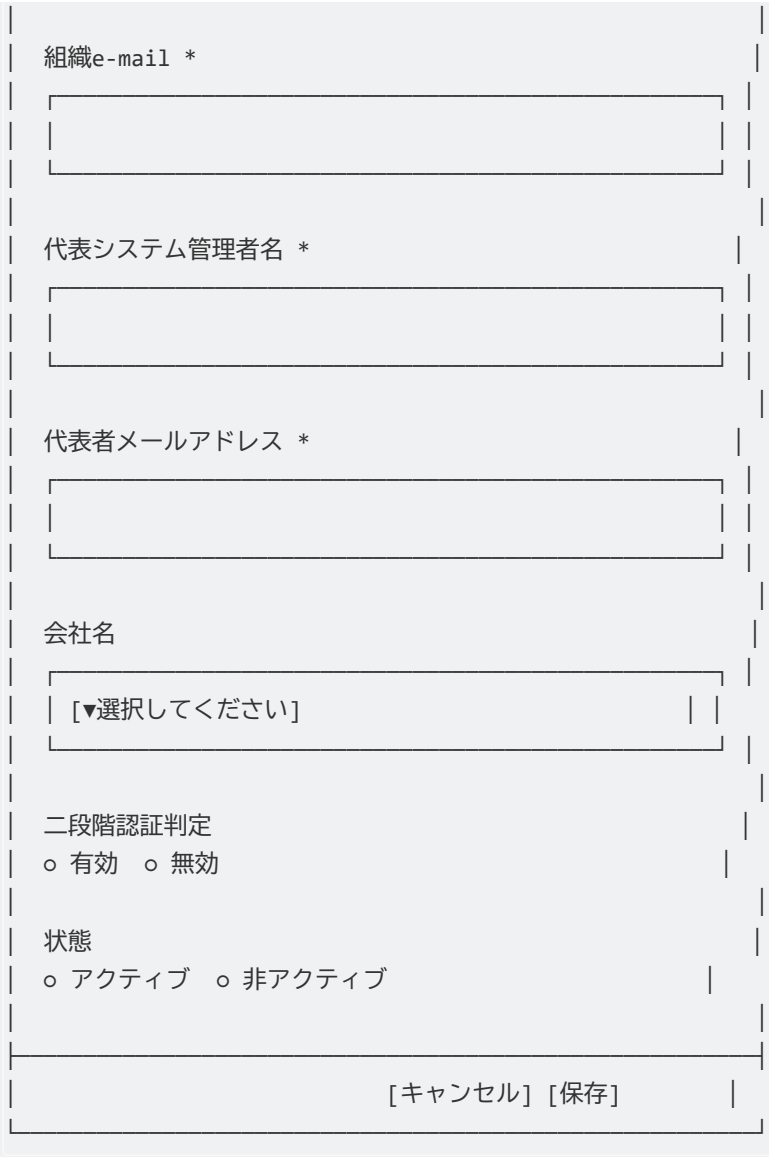
表示トリガー: 組織管理画面の「追加」ボタンクリック

モーダルタイプ: Modal

サイズ: 大（800px）

3.2. レイアウト





4. 入力項目

4.1. 組織CD（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「組織CD」
プレースホルダー	「組織CDを入力」
型	text

必須	○
最小文字数	1文字
最大文字数	50文字
許可文字	英数字、アンダースコア、ハイフン
重複チェック	有効

バリデーション:

```
{
  required: true,
  message: '組織CDを入力してください'
},
{
  min: 1,
  max: 50,
  message: '組織CDは1～50文字で入力してください'
},
{
  pattern: /^[a-zA-Z0-9_-]+$/,
  message: '組織CDは英数字、アンダースコア、ハイフンのみ使用できます'
}
```

リアルタイム重複チェック:

- 入力後500msのデバウンス
- 既存の組織CDと重複する場合はエラー表示
- 背景色を黄色（`colorWarningBg`）に変更

4.2. 組織表示名（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「組織表示名」
プレースホルダー	「組織表示名を入力」
型	text

必須	○
最小文字数	1文字
最大文字数	100文字

バリデーション:

```
{
  required: true,
  message: '組織表示名を入力してください'
},
{
  min: 1,
  max: 100,
  message: '組織表示名は1～100文字で入力してください'
}
```

4.3. 組織e-mail（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「組織e-mail」
プレースホルダー	「organization@example.com」
型	email
必須	○
最大文字数	100文字

バリデーション:

```
{
  required: true,
  message: '組織e-mailを入力してください'
},
{
  type: 'email',
  message: '有効なメールアドレスを入力してください'
},
```

```
{
  max: 100,
  message: '組織e-mailは100文字以内で入力してください'
}
```

4.4. 代表システム管理者名（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「代表システム管理者名」
プレースホルダー	「管理者名を入力」
型	text
必須	○
最大文字数	50文字

バリデーション:

```
{
  required: true,
  message: '代表システム管理者名を入力してください'
},
{
  max: 50,
  message: '代表システム管理者名は50文字以内で入力してください'
}
```

4.5. 代表者メールアドレス（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「代表者メールアドレス」

プレースホルダー	「admin@example.com」
型	email
必須	○
最大文字数	50文字

バリデーション:

```
{
  required: true,
  message: '代表者メールアドレスを入力してください'
},
{
  type: 'email',
  message: '有効なメールアドレスを入力してください'
},
{
  max: 50,
  message: '代表者メールアドレスは50文字以内で入力してください'
}
```

4.6. 会社名（任意）

コンポーネント: `<Form.Item>` + `<Select>`

仕様:

項目	値
ラベル	「会社名」
プレースホルダー	「選択してください」
型	select
必須	×
データソース	corporationテーブル

データ取得:

- API: `GET /api/v1/provider/corporation`
- レスポンス: `{ corporationId, corporationName }` の配列
- 表示: corporationName

- 値: corporationId

実装例:

```
<Form.Item label="会社名" name="corporationId">
  <Select
    placeholder="選択してください"
    allowClear
    options={corporations.map(corp => ({
      label: corp.corporationName,
      value: corp.corporationId
    }))}
  />
</Form.Item>
```

4.7. 二段階認証判定（必須）

コンポーネント: `<Form.Item>` + `<Radio.Group>`

仕様:

項目	値
ラベル	「二段階認証判定」
選択肢	有効(1) / 無効(0)
デフォルト	無効(0)
必須	○

実装例:

```
<Form.Item
  label="二段階認証判定"
  name="isEntry"
  initialValue={0}
  rules={[{ required: true, message: '二段階認証判定を選択してください' }]}
>
  <Radio.Group>
    <Radio value={1}>有効</Radio>
    <Radio value={0}>無効</Radio>
```

```
  </Radio.Group>
</Form.Item>
```

4.8. 状態（必須）

コンポーネント: `<Form.Item>` + `<Radio.Group>`

仕様:

項目	値
ラベル	「状態」
選択肢	アクティブ / 非アクティブ
デフォルト	アクティブ
必須	○

実装例:

```
<Form.Item
  label="状態"
  name="isActive"
  initialValue={true}
  rules={[{ required: true, message: '状態を選択してください' }]}
>
  <Radio.Group>
    <Radio value={true}>アクティブ</Radio>
    <Radio value={false}>非アクティブ</Radio>
  </Radio.Group>
</Form.Item>
```

5. アクションボタン

5.1. キャンセルボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「キャンセル」
タイプ	`default`
配置	フッター左側

動作:

1. 入力内容を破棄
2. ダイアログを閉じる
3. 親画面に戻る

5.2. 保存ボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「保存」
タイプ	`primary`
配置	フッター右側
ローディング	API呼び出し中は `true`
無効化	バリデーションエラー時

動作:

1. フォームバリデーション実行
2. 組織CD重複チェック
3. API呼び出し: `POST /api/v1/provider/organization`
4. 成功時:
    - 成功メッセージ表示（Toast）
    - 4つのテーブルへの自動挿入完了
        - organization（組織基本情報）
        - organization_user（代表システム管理者）
        - role（ROLE002: システム管理者）

        - role_permission（4つの機能権限）
    - ダイアログを閉じる
    - 親画面の一覧を再取得
5. 失敗時: エラーメッセージ表示

---

6. APIリクエスト仕様

6.1. エンドポイント

`POST /api/v1/provider/organization`

6.2. リクエストヘッダー

```
Content-Type: application/json
Authorization: Bearer {jwt_token}
```

6.3. リクエストボディ

```
{
  "organizationId": "550e8400-e29b-41d4-a716-446655440000",
  "organizationCd": "maiml_lab_002",
  "organizationDisplayName": "新研究所",
  "organizationEmail": "neworg@example.com",
  "chiefUserName": "田中太郎",
  "chiefUserEmail": "tanaka@example.com",
  "corporationId": "corp-uuid-001",
  "isEntry": 1,
  "isActive": true
}
```

注意: organizationIdはUUIDv4を生成して送信します。

6.4. レスポンス（成功時）


```
{
  "success": true,
  "data": {
    "organizationId": "550e8400-e29b-41d4-a716-446655440000",
    "organizationCd": "maiml_lab_002",
    "organizationDisplayName": "新研究所",
    "organizationEmail": "neworg@example.com",
    "chiefUserId": "tanaka@example.com",
    "chiefUserName": "田中太郎",
    "corporationName": "サンプル株式会社",
    "isEntry": 1,
    "isActive": true,
    "updatedAt": "20260219131500"
  },
  "meta": {
    "timestamp": "2026-02-19T13:15:00Z",
    "requestId": "req_abc123"
  }
}
```


6.5. レスポンス（エラー時）


```
{
  "success": false,
  "error": {
    "code": "ORGANIZATION_CD_DUPLICATE",
    "message": "この組織CDは既に使用されています",
    "details": []
  },
  "meta": {
    "timestamp": "2026-02-19T13:15:00Z",
    "requestId": "req_abc123"
  }
}
```


---

7. エラーメッセージ

7.1. バリデーションエラー

エラー条件	メッセージ
組織CDが空	「組織CDを入力してください」
組織CDが長すぎる	「組織CDは1〜50文字で入力してください」
組織CDに不正な文字	「組織CDは英数字、アンダースコア、ハイフンのみ使用できます」
組織CDが重複	「この組織CDは既に使用されています」
組織表示名が空	「組織表示名を入力してください」
組織表示名が長すぎる	「組織表示名は1〜100文字で入力してください」
組織e-mailが空	「組織e-mailを入力してください」
組織e-mailが無効	「有効なメールアドレスを入力してください」
組織e-mailが長すぎる	「組織e-mailは100文字以内で入力してください」
代表システム管理者名が空	「代表システム管理者名を入力してください」
代表システム管理者名が長すぎる	「代表システム管理者名は50文字以内で入力してください」
代表者メールアドレスが空	「代表者メールアドレスを入力してください」
代表者メールアドレスが無効	「有効なメールアドレスを入力してください」
代表者メールアドレスが長すぎる	「代表者メールアドレスは50文字以内で入力してください」

7.2. APIエラー

エラーコード	メッセージ
`ORGANIZATION_CD_DUPLICATE`	「この組織CDは既に使用されています」
`VALIDATION_ERROR`	「入力内容に誤りがあります」
`UNAUTHORIZED`	「この操作を実行する権限がありません」
`INTERNAL_SERVER_ERROR`	「サーバーエラーが発生しました。しばらく経ってから再試行してください」

8. 実装例（TypeScript + Next.js + Ant Design）

```
// frontend/src/components/Provider/OrganizationAddDialog.tsx
'use client';
```

```tsx
import React, { useState, useEffect } from 'react';
import { Modal, Form, Input, Select, Radio, Button, message, theme } from 'antd';
import apiClient from '@/utils/apiClient';
import { v4 as uuidv4 } from 'uuid';

const { useToken } = theme;

interface OrganizationAddDialogProps {
  open: boolean;
  onClose: () => void;
  onSuccess: () => void;
}

interface OrganizationFormValues {
  organizationCd: string;
  organizationDisplayName: string;
  organizationEmail: string;
  chiefUserName: string;
  chiefUserEmail: string;
  corporationId?: string;
  isEntry: number;
  isActive: boolean;
}

interface Corporation {
  corporationId: string;
  corporationName: string;
}

export default function OrganizationAddDialog({
  open,
  onClose,
  onSuccess
}: OrganizationAddDialogProps) {
  const { token } = useToken();
  const [form] = Form.useForm();
  const [isLoading, setIsLoading] = useState(false);
  const [isDuplicateChecking, setIsDuplicateChecking] = useState(false);
  const [isDuplicate, setIsDuplicate] = useState(false);
  const [corporations, setCorporations] = useState<Corporation[]>([]);

  // 会社一覧取得
  useEffect(() => {
    if (open) {
      fetchCorporations();
    }
  }, [open]);

  const fetchCorporations = async () => {
```

```
  try {
    const response = await apiClient.get('/api/v1/provider/corporation');
    if (response.data.success) {
      setCorporations(response.data.data);
    }
  } catch (error) {
    console.error('Failed to fetch corporations:', error);
  }
};

// 組織CD重複チェック（デバウンス付き）
useEffect(() => {
  const organizationCd = form.getFieldValue('organizationCd');
  if (!organizationCd || organizationCd.length === 0) {
    setIsDuplicate(false);
    return;
  }

  const timer = setTimeout(async () => {
    setIsDuplicateChecking(true);
    try {
      const response = await apiClient.get(
        `/api/v1/provider/organization/${organizationCd}`
      );
      // 既に存在する場合
      if (response.data.success) {
        setIsDuplicate(true);
        form.setFields([
          {
            name: 'organizationCd',
            errors: ['この組織CDは既に使用されています']
          }
        ]);
      }
    } catch (error: any) {
      // 404の場合は重複なし
      if (error.response?.status === 404) {
        setIsDuplicate(false);
      }
    } finally {
      setIsDuplicateChecking(false);
    }
  }, 500);

  return () => clearTimeout(timer);
}, [form.getFieldValue('organizationCd'), form]);

const handleSubmit = async (values: OrganizationFormValues) => {
  if (isDuplicate) {
    message.error('組織CDが重複しています');
```

```
      return;
    }

    setIsLoading(true);
    try {
      const response = await apiClient.post(
        '/api/v1/provider/organization',
        {
          organizationId: uuidv4(),
          organizationCd: values.organizationCd,
          organizationDisplayName: values.organizationDisplayName,
          organizationEmail: values.organizationEmail,
          chiefUserName: values.chiefUserName,
          chiefUserEmail: values.chiefUserEmail,
          corporationId: values.corporationId || null,
          isEntry: values.isEntry,
          isActive: values.isActive
        }
      );

      if (response.data.success) {
        message.success('組織を追加しました');
        form.resetFields();
        onSuccess();
        onClose();
      }
    } catch (error: any) {
      const errorMessage =
        error.response?.data?.error?.message || '組織の追加に失敗しました';
      message.error(errorMessage);
    } finally {
      setIsLoading(false);
    }
  };

  const handleCancel = () => {
    form.resetFields();
    setIsDuplicate(false);
    onClose();
  };

  return (
    <Modal
      title="組織追加"
      open={open}
      onCancel={handleCancel}
      footer={null}
      width={800}
      destroyOnClose
    >
```

```
      <Form
        form={form}
        layout="vertical"
        onFinish={handleSubmit}
        initialValues={{ isEntry: 0, isActive: true }}
      >
        {/* 組織CD */}
        <Form.Item
          label="組織CD"
          name="organizationCd"
          rules={[
            { required: true, message: '組織CDを入力してください' },
            { min: 1, max: 50, message: '組織CDは1～50文字で入力してください' },
            {
              pattern: /^[a-zA-Z0-9_-]+$/,
              message: '組織CDは英数字、アンダースコア、ハイフンのみ使用できます'
            }
          ]}
          extra="最大50文字"
          validateStatus={isDuplicate ? 'error' : undefined}
        >
          <Input
            placeholder="組織CDを入力"
            maxLength={50}
            style={{
              backgroundColor: isDuplicate ? token.colorWarningBg : undefined
            }}
            suffix={isDuplicateChecking ? <span>確認中...</span> : null}
          />
        </Form.Item>

        {/* 組織表示名 */}
        <Form.Item
          label="組織表示名"
          name="organizationDisplayName"
          rules={[
            { required: true, message: '組織表示名を入力してください' },
            { min: 1, max: 100, message: '組織表示名は1～100文字で入力してください' }
          ]}
        >
          <Input placeholder="組織表示名を入力" maxLength={100} />
        </Form.Item>

        {/* 組織e-mail */}
        <Form.Item
          label="組織e-mail"
          name="organizationEmail"
          rules={[
            { required: true, message: '組織e-mailを入力してください' },
            { type: 'email', message: '有効なメールアドレスを入力してください' },
```

```
      { max: 100, message: '組織e-mailは100文字以内で入力してください' }
    ]}
  >
    <Input placeholder="organization@example.com" maxLength={100} />
  </Form.Item>

  {/* 代表システム管理者名 */}
  <Form.Item
    label="代表システム管理者名"
    name="chiefUserName"
    rules={[
      { required: true, message: '代表システム管理者名を入力してください' },
      { max: 50, message: '代表システム管理者名は50文字以内で入力してください' }
    ]}
  >
    <Input placeholder="管理者名を入力" maxLength={50} />
  </Form.Item>

  {/* 代表者メールアドレス */}
  <Form.Item
    label="代表者メールアドレス"
    name="chiefUserEmail"
    rules={[
      { required: true, message: '代表者メールアドレスを入力してください' },
      { type: 'email', message: '有効なメールアドレスを入力してください' },
      { max: 50, message: '代表者メールアドレスは50文字以内で入力してください' }
    ]}
  >
    <Input placeholder="admin@example.com" maxLength={50} />
  </Form.Item>

  {/* 会社名 */}
  <Form.Item label="会社名" name="corporationId">
    <Select
      placeholder="選択してください"
      allowClear
      options={corporations.map(corp => ({
        label: corp.corporationName,
        value: corp.corporationId
      }))}
    />
  </Form.Item>

  {/* 二段階認証判定 */}
  <Form.Item
    label="二段階認証判定"
    name="isEntry"
    rules={[{ required: true, message: '二段階認証判定を選択してください' }]}
  >
    <Radio.Group>
```

```
            <Radio value={1}>有効</Radio>
            <Radio value={0}>無効</Radio>
          </Radio.Group>
        </Form.Item>

        {/* 状態 */}
        <Form.Item
          label="状態"
          name="isActive"
          rules={[{ required: true, message: '状態を選択してください' }]}
        >
          <Radio.Group>
            <Radio value={true}>アクティブ</Radio>
            <Radio value={false}>非アクティブ</Radio>
          </Radio.Group>
        </Form.Item>

        {/* フッター */}
        <Form.Item style={{ marginBottom: 0, marginTop: 24 }}>
          <div style={{ display: 'flex', justifyContent: 'flex-end', gap: 8 }}>
            <Button onClick={handleCancel}>キャンセル</Button>
            <Button
              type="primary"
              htmlType="submit"
              loading={isLoading}
              disabled={isDuplicate || isDuplicateChecking}
            >
              保存
            </Button>
          </div>
        </Form.Item>
      </Form>
    </Modal>
  );
}
```

---

9. テスト要件

9.1. 単体テスト

テストケース

- バリデーション機能のテスト

- 重複チェック機能のテスト
- フォーム送信処理のテスト
- 会社一覧取得のテスト

9.2. 統合テスト

テストシナリオ

1. 正常な組織追加
2. 組織CD重複エラー
3. 必須項目未入力エラー
4. 最大文字数超過エラー
5. 不正な文字列入力エラー
6. メールアドレス形式エラー
7. 会社選択（任意項目）
8. 4テーブル自動挿入の確認

---

10. アクセシビリティ

10.1. キーボード操作

- Tab: 次の入力項目へフォーカス移動
- Shift + Tab: 前の入力項目へフォーカス移動
- Enter: フォーム送信
- Escape: ダイアログを閉じる

10.2. スクリーンリーダー対応

- 各入力項目に適切なラベルを設定
- エラーメッセージを `role="alert"` で通知
- 必須項目を `aria-required="true"` で明示

11. 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-02-19	初版作成	Development Team